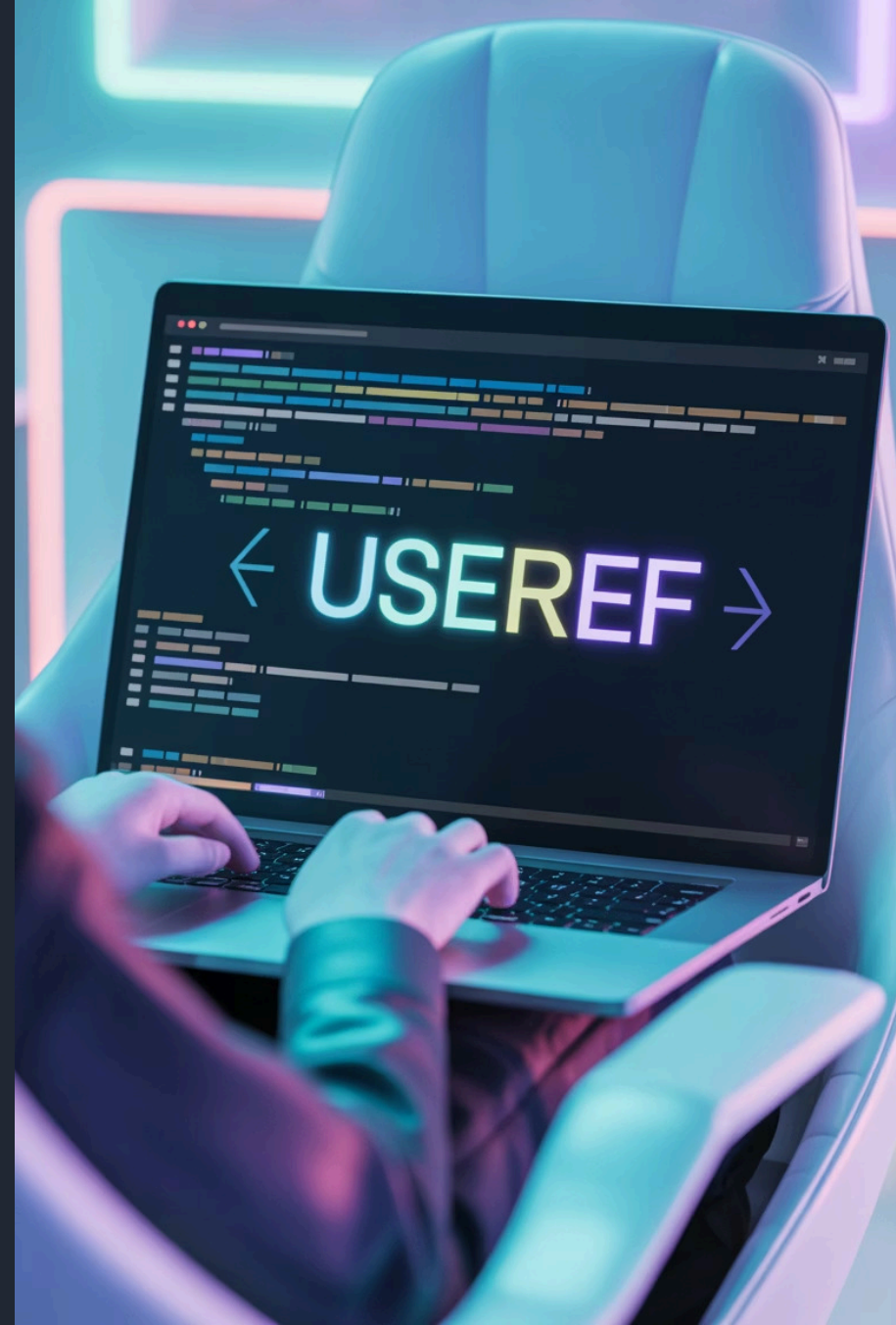


Persistência e Lógica de UI

Vamos explorar dois conceitos fundamentais para dominar este hook poderoso: a persistência entre renderizações e o papel da lógica de UI. Estes conhecimentos são essenciais para desenvolvedores React que desejam otimizar seus componentes e fazer escolhas adequadas entre useRef e useState.



O que significa persistir entre renderizações

Comportamento padrão do React

Sempre que o estado de um componente muda, o React **reexecuta a função inteira do componente**, o que causa a **recriação de todas as variáveis normais** dentro dela.

Problema com variáveis comuns:

```
function MeuComponente() {  
  let contador = 0;  
  
  function somar() {  
    contador++;  
    console.log(contador);  
  }  
  
  return <button onClick={somar}>Clique</button>;  
}
```

Cada clique no botão, o valor de contador volta para 0, pois a variável **não persiste** entre renderizações.

Solução: useRef para persistência

```
import { useRef } from 'react';  
  
function MeuComponente() {  
  const contador = useRef(0);  
  
  function somar() {  
    contador.current++;  
    console.log(contador.current);  
  }  
  
  return <button onClick={somar}>Clique</button>;  
}
```

Com useRef, contador.current **mantém seu valor entre renderizações**. O componente pode reexecutar, mas a referência continua viva.

Importante: useState também persiste valores, mas **dispara re-renderizações**, enquanto useRef não causa novas renderizações.

USESTATE

```
ud.ee:"5510y>  
ud.ee".esstaetf—"310y>  
ud.aet".eestaetf—"310y>  
uid.es".eset.eet—" 7513y>  
uid.se".eestvct—" 2513y>
```

USEREF

```
ud.ee"7330y>  
ud.ee".ectreet—" 7513y>  
ud.ee".esstaet—" 7510y>  
ud.ee".esctet—" 7510y>  
ud.ee".eastee—" 7510y>
```

O que é lógica de UI

Lógica de UI (User Interface) é qualquer informação que, ao mudar, precisa ser refletida na interface visual da aplicação.

1

Exemplo com useState

```
const [nome, setNome] = useState("");  
  
<input onChange={(e) => setNome(e.target.value)} />  
<p>Olá, {nome}!</p>
```

Aqui, nome **faz parte da lógica de UI**. Quando o usuário digita, o componente **re-renderiza** para mostrar o novo nome na tela.

2

Exemplo com useRef

```
const nomeRef = useRef("");  
  
<input onChange={(e) => nomeRef.current = e.target.value} />  
<p>Olá, {nomeRef.current}!</p> // Não atualiza  
automaticamente
```

nomeRef.current muda, mas **a interface não reage**. Isso porque useRef **não causa re-renderizações**.

Comparação entre useRef e useState

Característica	useRef	useState
Persiste entre renderizações	✓ Sim	✓ Sim
Causa re-renderização	✗ Não	✓ Sim
Ideal para lógica de UI	✗ Não	✓ Sim
Ideal para lógica imperativa	✓ Sim	✗ Não recomendado

Quando usar useRef?

Use para operações que não precisam refletir na interface do usuário, como acessar elementos DOM, armazenar valores anteriores, ou manter contadores internos.

Quando usar useState?

Use quando a mudança de valor precisa ser visualizada pelo usuário, como formulários, toggles, contadores visíveis ou qualquer dado que precise ser renderizado.

Quando usar cada um?

1

Situações para useRef

- Focar em um elemento input
- Controlar animações imperativas
- Ler posição de scroll
- Armazenar valores anteriores
- Manter temporizadores (setTimeout/setInterval)
- Guardar flags internas que não afetam a UI

```
const inputRef = useRef(null);  
// Mais tarde...  
inputRef.current.focus();
```

2

Situações para useState

- Exibir dados dinâmicos na tela
- Controlar valores de formulários
- Alternar visibilidade de elementos
- Aplicar classes CSS dinamicamente
- Gerenciar estados de componentes (loading, erro)
- Controlar temas e preferências de usuário

```
const [tema, setTema] = useState('claro');  
// Quando o tema muda, a UI atualiza
```

A regra de ouro: se a informação precisa aparecer na tela ou influenciar o que é renderizado, use useState. Se não, considere useRef.



Conclusão: Escolhendo o hook certo

useRef

Não renderiza o componente, mas guarda valores entre execuções. É útil quando você precisa persistência sem impacto visual.

useState

Ideal quando você quer controlar e atualizar a interface do usuário. Causa re-renderizações para refletir mudanças visuais.

Lembre-se:

- A escolha entre useRef e useState não é sobre preferência pessoal, mas sobre a natureza do dado que você está manipulando
- Use o hook certo para otimizar o desempenho do seu aplicativo React
- Combine os dois hooks para criar componentes mais eficientes e responsivos

Na próxima aula, exploraremos casos práticos e avançados de useRef com exemplos reais de projetos.